Reg N° : 3250070061

Computer and software Engineering
Mobile Application Systems and design
27 February 2026

Question 1 : State Management in Flutter

- Provider : is a wrapped around flutter's inheritedWidget that makes it easier to manage and share across the widget tree.
  It uses a changeNotifier class to hold state and notifies listening widgets to rebuild whenever the state changes. It is simple, Lightweight and officially recommended by flutter team for most use cases.

- Riverpod : is an improved and more powerful version of provider, unlike provider it does not depend on widget tree or buildContext, making it more flexible and easier to test. it is compile-safe because errors are caught at compile time rather than runtime

- Bloc (Business logic component) : is state management pattern that separates business logic from UI using streams. it works by receiving Events (user actions / inputs), processing them inside bloc and emitting states that UI reacts to. it follows strict architecture and is very structured

- Getx : is light weight and fast state management solution that handles dependency injection and routing in single solution. state is managed using reactive variables (Rx types) and get x controllers, making it very fast to develop with, though it is less strict in enforcing architecture.

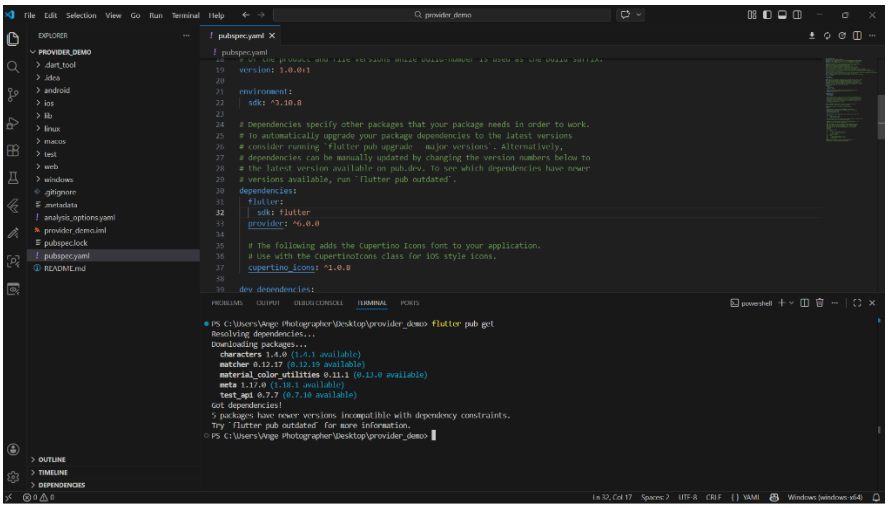
Question 2 : When to use each state Management

Situations	Provider	RiverPod	Bloc	Getx
Small applications	very Good	Good	Too complex	very Good
Medium applications	Good	very Good	Good	Good
Large / enterprise applications	Limited / gets complex	very Good	Excellent	Risky
Team Projects	Good	very Good	Excellent	Hard to maintain (depends)
Fast development	Good	Medium	slow	Excellent
Strict architecture requirement	Basic (Moderate)	Good	Excellent	too flexible (Not strict)

Bloc is more suitable for enterprise apps because it follows strict architecture
while,
Getx is better for fast and simple development

Question 3: Explain in detail the key steps on how Provider is used

- **Adding dependencies:** The provider package is added to pubspec.yaml and installed using flutter pub get, making it available throughout the project.

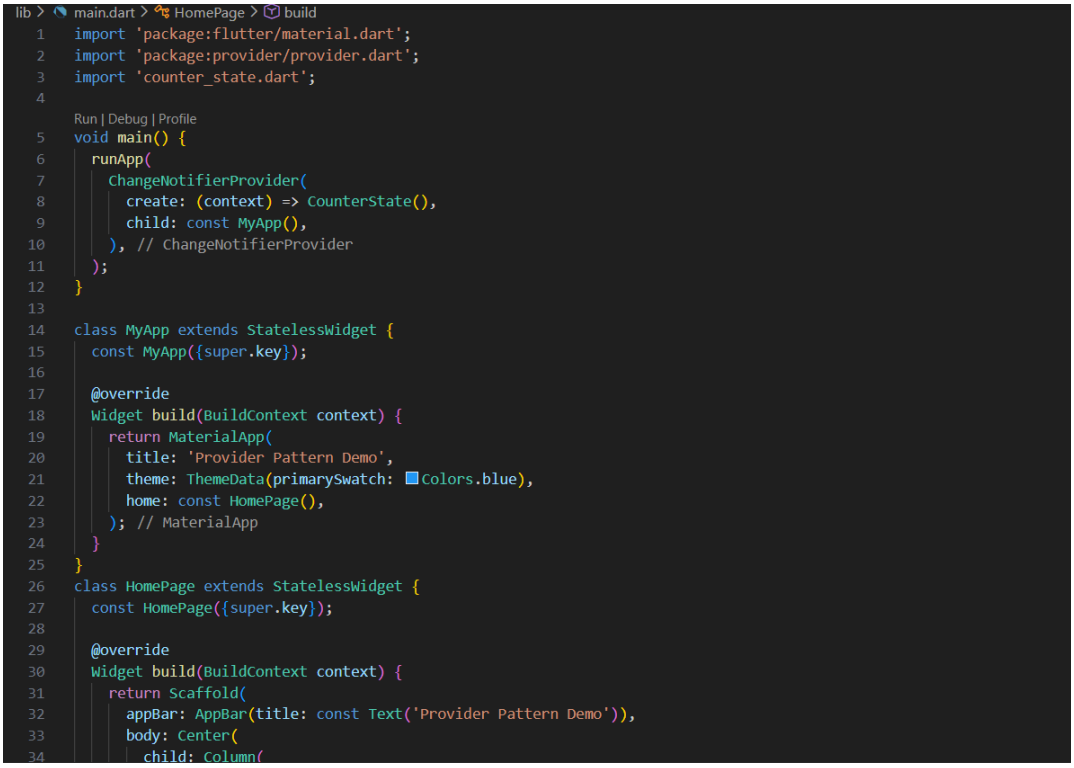


- **Creating a state class:** A Dart class extending ChangeNotifier is created to hold the app's data and logic. The notifyListeners() method is called after any state change to signal widgets to rebuild.

lib > counter_state.dart > CounterState

```dart
import 'package:flutter/foundation.dart';

class CounterState extends ChangeNotifier {
  int _counter = 0;

  int get counter => _counter;

  void increment() {
    _counter++;
    notifyListeners();
  }

  void decrement() {
    _counter--;
    notifyListeners();
  }

  void reset() {
    _counter = 0;
    notifyListeners();
  }
}
```

- **Providing a state:** The app is wrapped with ChangeNotifierProvider in main.dart, which creates a single instance of the state class and makes it accessible to all widgets in the widget tree.



```
lib > main.dart > HomePage > build
1  import 'package:flutter/material.dart';
2  import 'package:provider/provider.dart';
3  import 'counter_state.dart';
4
   Run | Debug | Profile
5  void main() {
6    runApp(
7      ChangeNotifierProvider(
8        create: (context) => CounterState(),
9        child: const MyApp(),
10     ), // ChangeNotifierProvider
11   );
12 }
13
14 class MyApp extends StatelessWidget {
15   const MyApp({super.key});
16
17   @override
18   Widget build(BuildContext context) {
19     return MaterialApp(
20       title: 'Provider Pattern Demo',
21       theme: ThemeData(primarySwatch: Colors.blue),
22       home: const HomePage(),
23     ); // MaterialApp
24   }
25 }
26 class HomePage extends StatelessWidget {
27   const HomePage({super.key});
28
29   @override
30   Widget build(BuildContext context) {
31     return Scaffold(
32       appBar: AppBar(title: const Text('Provider Pattern Demo')),
33       body: Center(
34         child: Column(
```

- **Accessing a state:** Widgets use context.watch<T>() to subscribe to the state. Any widget that watches the state will automatically rebuild whenever notifyListeners() is called.

```dart
class HomePage extends StatelessWidget {
  const HomePage({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Provider Pattern Demo')),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            // Method 1: Using Consumer
            Consumer<CounterState>(
              builder: (context, counterProvider, child) {
                return Text(
                  '${counterProvider.counter}',
                  style: const TextStyle(
                      fontSize: 48, fontWeight: FontWeight.bold), // TextStyle
                ); // Text
              },
            ), // Consumer

            // Method 2: Using context.watch()
            Text(
              '${context.watch<CounterState>().counter}',
              style: const TextStyle(fontSize: 36, fontWeight: FontWeight.bold),
            ), // Text

            // Method 3: Using Provider.of
            Text(
              '${Provider.of<CounterState>(context, listen: true).counter}',
              style: const TextStyle(fontSize: 36, fontWeight: FontWeight.bold),
            ), // Text
```

- **Updating a state:** Widgets use context.read<T>() to call methods on the state class without subscribing to changes. This is used in button actions and event handlers to trigger state updates.

```dart
            Row(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                ElevatedButton.icon(
                  onPressed: () => context.read<CounterState>().decrement(),
                  icon: const Icon(Icons.remove),
                  label: const Text('Decrease'),
                  style: ElevatedButton.styleFrom(backgroundColor: const Color.fromARGB(255, 241, 244, 54)),
                ), // ElevatedButton.icon
                const SizedBox(width: 10),
                ElevatedButton.icon(
                  onPressed: () => context.read<CounterState>().increment(),
                  icon: const Icon(Icons.add),
                  label: const Text('Increase'),
                  style: ElevatedButton.styleFrom(backgroundColor: const Color.fromARGB(255, 76, 119, 175)),
                ), // ElevatedButton.icon
                const SizedBox(width: 10),
                ElevatedButton.icon(
                  onPressed: () => context.read<CounterState>().reset(),
                  icon: const Icon(Icons.refresh),
                  label: const Text('Reset'),
                  style: ElevatedButton.styleFrom(backgroundColor: const Color.fromARGB(255, 255, 0, 132)),
                ), // ElevatedButton.icon
              ],
            ), // Row

          ],
        ), // Column
      ), // Center
    ); // Scaffold
  }
}
```

- **How UI build happens:** When a button is tapped, notifyListeners() fires and only widgets using Consumer or watch rebuild — not the entire app

```dart
// STEP 6: HOW UI REBUILD HAPPENS
Container(
  padding: const EdgeInsets.all(15),
  decoration: BoxDecoration(
    border: Border.all(color: Colors.blue, width: 2),
    borderRadius: BorderRadius.circular(8),
  ), // BoxDecoration
  child: Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      const Text(
        'How UI Rebuild Happens:',
        style: TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
      ), // Text
      const SizedBox(height: 10),
      const Text(
        '1. User presses button (e.g., Increase)\n'
        '2. context.read<CounterState>().increment() is called\n'
        '3. _count++ runs and notifyListeners() is called\n'
        '4. Provider notifies all registered listeners\n'
        '5. Widgets using Consumer/watch are marked for rebuild\n'
        '6. Flutter calls build() on ONLY those widgets\n'
        '7. UI updates with new counter value\n'
        '8. Widgets using read() do NOT rebuild',
        style: TextStyle(fontSize: 12, height: 1.6),
      ), // Text
    ],
  ), // Column
), // Container

const SizedBox(height: 30),
```